

Development and Explanation

config/ – Configuration Files

- **connect.js** – Establishes a secure **MongoDB database connection** using **Mongoose**, ensuring reliable communication between the application and the database.
-

controllers/ – Business Logic Layer

- **adminController.js** – Handles **administrator operations** such as **user management**, **property approval**, **booking management**, and **platform analytics**.
 - **ownerController.js** – Manages **landlord functionalities**, including **adding**, **editing**, **deleting**, and **managing property listings** along with booking requests.
 - **userController.js** – Implements **user authentication**, **profile management**, **property browsing**, **wishlist management**, and **property booking** functionalities.
-

middlewares/ – Middleware Components

- **authMiddleware.js** – Performs **JWT token authentication**, secures **protected routes**, verifies user identity, and implements **role-based authorization** for **Admin**, **Landlord**, and **User** access.
-

models/ – Database Schemas

- **BookingSchema.js** – Defines the **booking structure**, including **tenant details**, **property information**, **visit date**, **booking status**, and **booking messages**.
 - **PropertySchema.js** – Stores complete **property information**, including **location**, **price**, **property type**, **amenities**, **owner details**, and **listing status**.
 - **UserSchema.js** – Defines user information such as **name**, **email**, **password**, **role**, **contact details**, and **authentication credentials**.
-

routes/ – API Routing Layer

- **adminRoutes.js** – Defines **Admin API endpoints** for **user management**, **property moderation**, **analytics**, and **booking administration**.
- **ownerRoutes.js** – Provides **Landlord API routes** for **property management**, **image uploads**, and **booking request handling**.

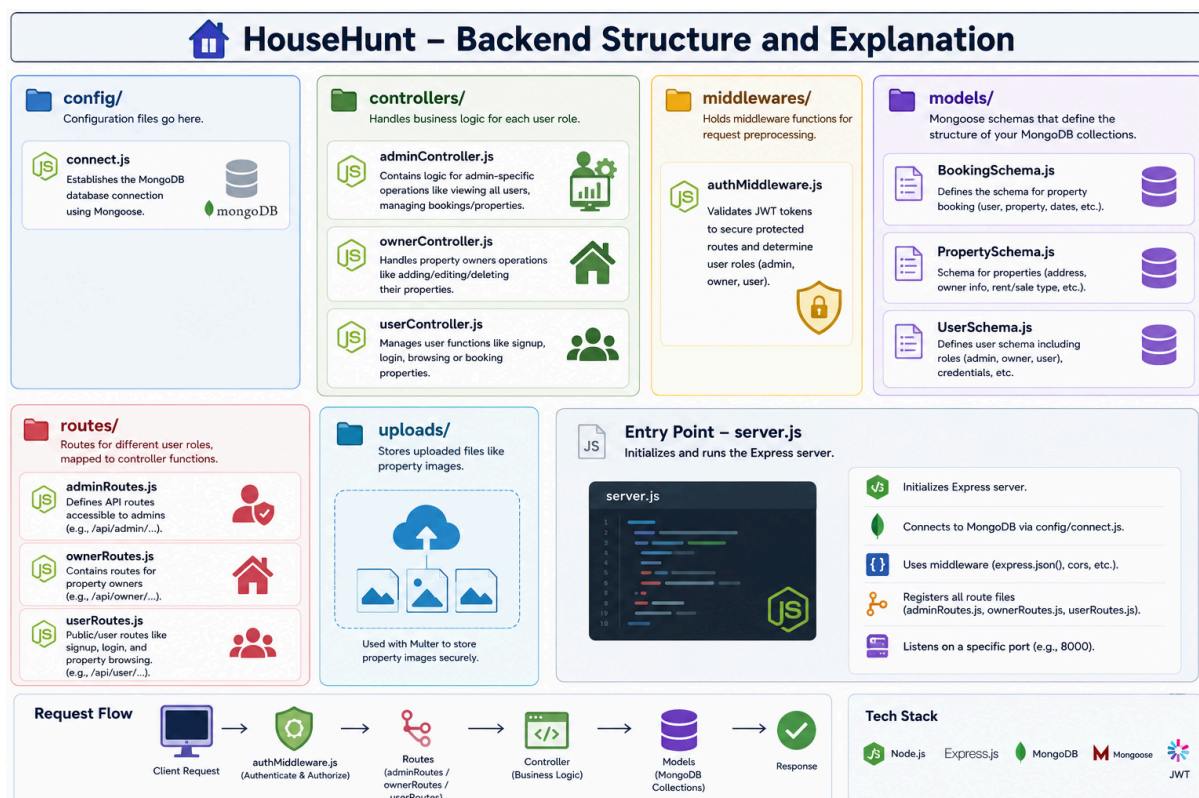
- **userRoutes.js** – Contains **public** and **authenticated user routes** for **registration, login, property browsing, wishlist, and booking operations**.

uploads/ – File Storage

- Stores uploaded **property images** using **Multer**, enabling landlords to upload and manage images associated with property listings.

Entry Point – server.js

- **server.js** serves as the **main entry point** of the backend application. It initializes the **Express.js server**, establishes the **MongoDB connection**, configures essential **middleware** such as **CORS, JSON parsing, and authentication**, registers all **API routes**, and starts the server on the configured **port**, making the application ready to handle client requests.



User Request



Express Server
(server.js)



